第5章 第5节 向量数据库（一）- 核心算法

- 来源: https://class.imooc.com/lesson/5272#mid=104917
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: 向量数据库
- 课节: 图文 1-1 向量数据库（一）- 核心算法

1-1 向量数据库（一）- 核心算法

第5章 第5节 向量数据库（一）- 核心算法

> [!TIP] 阅读指南 上一节学会了通过 API 获取 Embedding 向量——给文字贴上了"语义坐标"。但有了向量之后呢？ 假设知识库里有成千上万条文档，每条都变成了高维向量。用户提一个问题，需要从海量向量中找出最相关的几条。一个个比对效率极低。这一节解决的就是这个核心问题。

5.1 为什么需要向量数据库

从一个真实场景说起。

一个现实的难题

假设做一个企业知识库问答系统。先看数据规模：

- 知识库有1000份文档
- 每份文档平均5页，切分成100个片段（Chunk）
- 算下来：10万个文档片段

10万个文档片段

- 每个片段都转成了1536维向量

1536维向量

现在用户问：“年假怎么申请？”

系统需要做什么？

- 把这个问题也转成向量
- 在10万个向量中找到最相似的5个
- 把这5个片段交给大模型，让它生成答案

第2步是关键。怎么在10万个向量里快速找到最相似的那5个？

怎么在10万个向量里快速找到最相似的那5个？

朴素方法的困境

最直接的想法是什么？“暴力搜索”——一个个比对。

```
# 伪代码：朴素的相似度搜索
question_vec = get_embedding("年假怎么申请？")

similarities = []
for i, doc_vec in enumerate(all_100k_vectors):
    # 计算余弦相似度
    sim = cosine_similarity(question_vec, doc_vec)
    similarities.append((sim, i))

# 排序，取前5个
top_5 = sorted(similarities, reverse=True)[:5]
```

这段代码有什么问题？

算笔账：

- 要计算10万次向量点积（每次都是1536维的计算）
- 还要对10万个结果排序
- 单次查询耗时（预估）：约2-3秒

2-3秒什么概念？想想这些场景：

- 用户会觉得卡顿（现在的人哪有这个耐心）
- 100个用户同时问问题？系统直接崩溃
- 如果用户增长到1万个和10万个呢？

在真实的生产环境中，这个问题会被成倍放大：

- [×] 100万条文档 → 30秒查询时间

- [×] 1000万条文档 → 5分钟查询时间
- [×] 用户等不了，系统也撑不住

这就是为什么需要向量数据库。

5.2 向量数据库的价值

向量数据库做了件很聪明的事：它不会每次都傻傻地遍历所有向量，而是提前把向量组织成特殊的数据结构，让检索可以"跳着走"。

打个比方。

朴素搜索 = 在字典里逐页翻查

> 要找"Apple"这个词 从第1页开始，一页页翻，直到找到 结果：翻了100页才找到

向量数据库 = 用索引和目录

> 要找"Apple" 看首字母索引 → 直接跳到"A"开头的部分 看第二个字母 → 跳到"Ap" 看第三个字母 → 定位到"App" 找到"Apple" 翻页次数：只需3-5次

这不是快一点点，是快到离谱。向量数据库的核心优势就在这：

- [√] 速度快：毫秒级响应，支持高并发

速度快

- [√] 可扩展：百万、千万、亿级数据都能处理

可扩展

- [√] 准确率高：虽然是"近似搜索"，但准确率通常在99%以上

准确率高

- [√] 工程化：支持增删改查、批量操作、分布式部署

工程化

5.3 向量数据库的工作原理

向量数据库的核心技术叫近似最近邻搜索（Approximate Nearest Neighbor，ANN）。名字听起来有点拗口，但思想很简单。

核心思想：空间分区

最好理解的方法就是"空间分区"——把向量空间划分成多个区域，查询时只需要在最相关的几个区域里搜索。

用生活中的例子来理解。

5.4 类比：在体育场找人

想象在一个能容纳10万人的体育场里，要随便找到一位穿红色衣服的观众。怎么找?

暴力搜索

> 从第1排第1座开始，按座位顺序一个个检查每个人的衣服颜色， 直到终于遇到一位穿红衣服的观众。 如果平均检查每个人需要 1 秒，想靠这种方式查一遍，这得看运气，最长需要30个小时左右。

分区搜索（IVF 的思路）

> 先把体育场划分成100个看台区域； 快速扫一眼每个区域的大致颜色分布， 找出"红衣服明显更多"的3个区域； 只在这3个区域里逐排逐座细查。 这样只需要仔细检查几千个人，耗时从"几个小时"降到"几十分钟"。

图搜索（HNSW 的思路）

> 从体育场入口的引导员开始打听； 引导员说： “我刚才看到A区红衣服比较多，去A区问问” ； 到了A区后，当地引导员再指"A3看台那一片红衣服最多"； 走到A3，看台管理员直接指"就在这边第5排"； 几分钟内，就锁定到了那一小撮红衣服观众中的一位。 整个过程只和少数几个"关键节点"打了交道，而不是亲自看过10万人。

向量数据库就是用类似的思想，把高维空间中的向量组织起来。下面看三种主流的索引算法。了解即可。

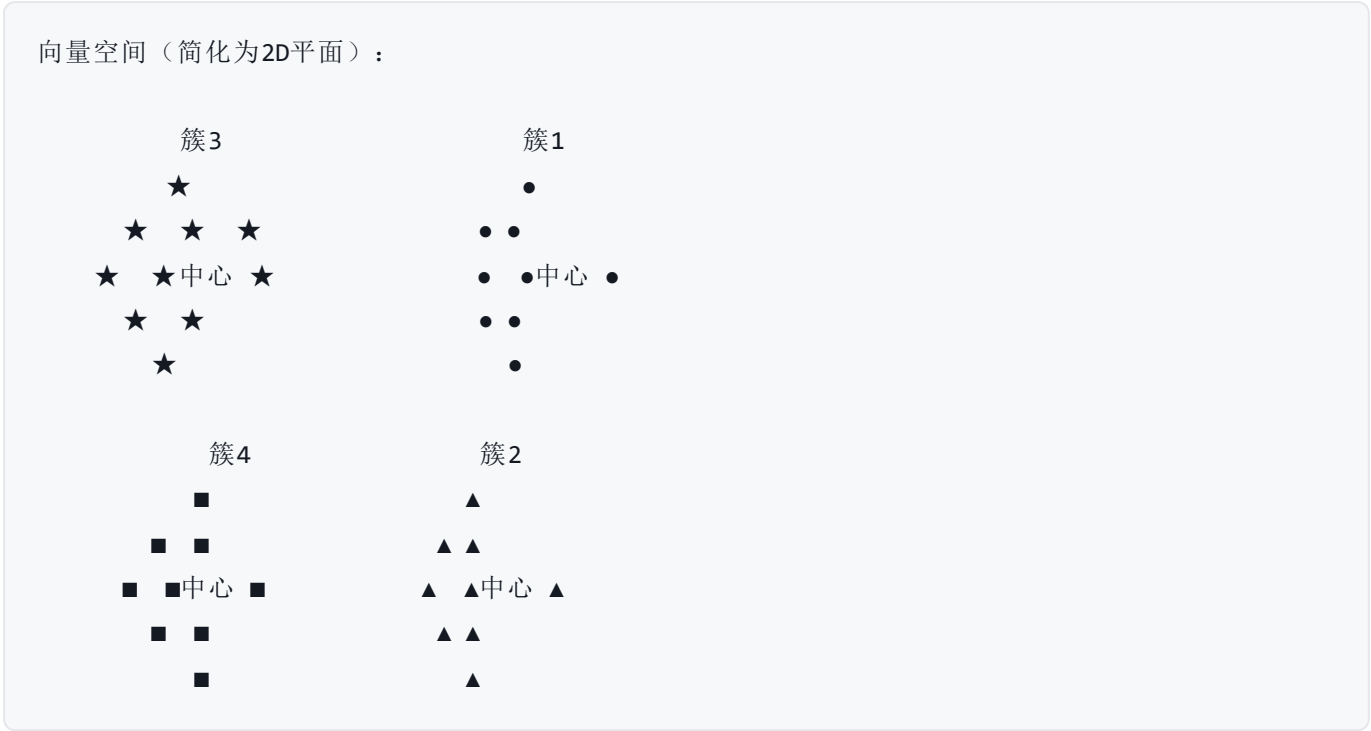
5.5 IVF（倒排文件索引）

核心思想：聚类 + 倒排

第1步：聚类

- 使用 K-Means 算法，把所有向量按距离聚成 N 个簇（比如 N=1000）
- 算法会找到 N 个“簇中心”，每个簇内的向量距离都比较近
- 比如：把 10 万个点分成 1000 组，语义相近的向量会聚在一起

图示：



解读：

- 每个符号（●▲■★）代表一个向量
- 同一种符号 = 同一个簇（逻辑分组）
- “中心” = 簇中心，位于该簇所有向量的几何中心
- 语义相近的向量聚在一起，距离远的分开

第2步：查询

- 计算查询向量到各个簇中心的距离
- 找到距离最近的 K 个簇（比如 K=10）
- 只在这 K 个簇里搜索，而不是搜索全部 10 万个向量

只在这 K 个簇里搜索

- 返回最相似的结果

用一个示例来理解。假设要找"Python 语言怎么样？"的相关内容：

> 步骤1：计算查询向量到所有1000个簇中心的距离 簇1(编程语言): 距离 = 0.2 ← 最近 簇2(美食菜谱): 距离 = 0.9 簇3(旅游攻略): 跨离 = 0.85 ... 簇15(技术教程): 距离 = 0.18 ← 最近 ... 步骤2：选择最近的10个簇 簇15(技术教程)、簇1(编程语言)、簇8(软件开发)... 这些簇共有 1000 个向量 步骤3：只在这 1000 个向量中搜索 找到： “Python 入门教程：基础语法” (距离 = 0.10) “Python 与其他语言对比” (距离 = 0.13) “Python 应用场景分析” (距离 = 0.16) 结果：返回这 3 条相关内容

5.6 HNSW（层次化可导航小世界图）

核心思想：多层图结构 + 贪婪搜索

HNSW把向量组织成一个多层的图结构。

整体结构

```
第2层（稀疏）：只有少数节点
    A --------------------------- D

第1层（中等）：节点增多
    A ------- B ------- D
              |         |
              C ------- E

第0层（密集）：包含所有节点
    A - B - D - F
    |   |   |   |
    H - C - E - G
```

查询流程

- 从顶层开始，在每一层找到离查询向量最近的点
- 逐层下降，每一层都更精确地逃近目标
- 最终在底层找到最相似的向量

查找示例

假设要找"Python 数据分析"的相关内容

> 步骤1：第2层（稀疏，只有10个点） 当前在入口点 A 计算查询向量“Python 数据分析”到节点的距离： 到 A(前端开发): 0.8 到 D(后端框架): 0.5 D 更近 → 跳到 D 步骤2：第1层（中等，有50个点） 当前在点 D 计算查询向量到 D 的邻居的距离： 到 A(前端开发): 0.8 到 B(Web开发): 0.4 到 E(Python教程): 0.3 E 更近 → 跳到 E 步骤3：第0层（密集，所有100个点） 当前在点 E 计算查询向量到 E 的邻居的距离： 到 C(Java开发): 0.6 到 D(后端框架): 0.5 到 G(Python数据分析教程): 0.15 G 更近 → 跳到 G 计算查询向量到 G 的邻居的距离： 到 E(Python教程): 0.3 到 F(Python编程): 0.4 到 H(NumPy库): 0.2 G 自己(0.15)比所有邻居都近 → 停止 结果：找到 G - “Python数据分析教程”

上述流程最终能找到最相似的"邻近"向量，但不只想找1个"最"相近的，而是想找多个相近的（Top-K）向量。 比如如果要找 Top-5 最相似的结果，那按照上述的流程做5遍吗？并不是。真正的方法如下：

- 搜索过程中维护一个“候选集”（比如保留搜索过的前10个最近的点）
- 搜索结束后，从候选集中返回距离最近的5个

> [!NOTE] 向量怎么分层？ 上述图的查找方法，是建立在分层的基础上。一堆向量是怎么分层的？有兴趣可以阅读下。 其实很简单，完全是随机分层的。 HNSW 建索引时，每个向量都会随机分配一个“层级”： 每个向量插入时，随机决定它的“最高层级”（就像抛硬币） 大多数向量只到第0层，少数能到第1层，更少数到第2层 就像金字塔：底层人多，越往上人越少 具体例子： 向量 A：随机到“第2层” → 在第0、第1、第2层都有（作为“快速跳转点”） 向量 B：随机到“第1层” → 在第0、第1层有，第2层没有 向量 C：随机到“第0层” → 只在第0层（大多数向量都这样） 为什么要随机？ 随机分层能保证图结构的平衡性，避免“热点聚集” 不需要复杂的计算，构建索引更快 实践证明：随机分层的效果非常好！ 这样设计的好处： 顶层节点少 → 搜索快速定位大致区域 底层节点全 → 精确找到最相似的

5.7 ■ 学点英语

中文	English	音标	说明
近似近邻	Approximate Nearest Neighbor (ANN)	/əˈprɑːksɪmət ˈnɪərəst ˈneɪbər/	通过索引结构牺牲少量精度换取极快检索速度的算法
精确近邻	K-Nearest Neighbor (KNN)	/keɪ ˈnɪərəst ˈneɪbər/	遍历全部数据确保返回最精确的Top-K结果
倒排文件索引	Inverted File Index (IVF)	/ɪnˈvɜːrtɪd faɪl ˈɪndeks/	通过聚类将向量空间分区，只搜索最近簇的索引方法
分层可导航小世界图	Hierarchical Navigable Small World (HNSW)	/ˌhaɪəˈrɑːrkɪkl ˈnævɪgəbl smɔːl wɜːrld/	基于多层图结构的近似搜索算法，查询极快
Voronoi 单元	Voronoi Cell	/vəˈroʊnɔɪ sel/	IVF聚类后向量空间被划分成的区域，查询时搜索最近单元